El concepto más importante en cualquier sistema operativo es el de **proceso, una abstracción de un programa en ejecución.**
Los procesos son una de las abstracciones más antiguas e importantes que proporcionan los sistemas operativos: proporcionan la capacidad de operar (pseudo) concurrentemente, incluso cuando hay sólo una CPU disponible. Convierten una CPU en varias CPU virtuales.

2.1 PROCESOS

Todas las computadoras modernas ofrecen **varias cosas al mismo tiempo.** En cualquier sistema de multiprogramación, la CPU **conmuta de un proceso a otro con rapidez,** ejecutando cada uno durante décimas o centésimas de milisegundos: hablando en sentido estricto, en cualquier instante la CPU está ejecutando sólo un proceso, y en el transcurso de 1 segundo podría trabajar en varios de ellos, dando la apariencia de un **paralelismo** (o pseudoparalelismo, para distinguirlo del verdadero paralelismo de hardware de los sistemas multiprocesadores con dos o más CPUs que comparten la misma memoria física).
Un proceso no es más que una instancia de un programa en ejecución, incluyendo los valores actuales del contador de programa, los registros y las variables. Esta **conmutación** rápida de un **proceso a otro** se conoce como **multiprogramación**.

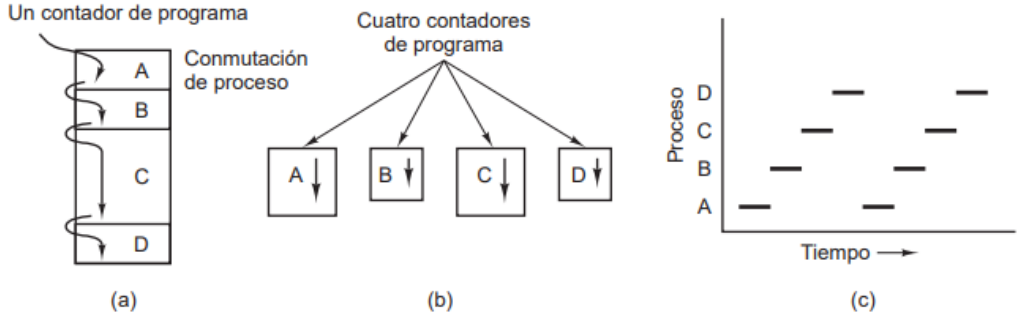


Figura 2-1. (a) Multiprogramación de cuatro programas. (b) Modelo conceptual de cuatro procesos secuenciales independientes. (c) Sólo hay un programa activo a la vez.

La **diferencia** entre un proceso y un programa es sutil pero crucial. La idea clave es que un **proceso es una actividad de cierto tipo:** tiene un programa, una entrada, una salida y un estado. Varios procesos pueden compartir un solo procesador mediante el uso de un algoritmo de planificación.

Creación de un proceso

Hay cuatro eventos principales que provocan la creación de procesos:

1. El arranque del sistema.
2. La ejecución, desde un proceso, de una llamada al sistema para creación de procesos.
3. Una petición de usuario para crear un proceso.
4. El inicio de un trabajo por lotes.

Demonios: Los procesos que permanecen en segundo plano para manejar ciertas actividades como correo electrónico, páginas Web, noticias, impresiones, etcétera. En Unix podemos usar ps para listar los procesos en ejecución.

Además de los procesos que se crean al arranque, posteriormente se pueden crear otros. A menudo, un **proceso en ejecución** emitirá **llamadas al sistema** para **crear** uno o más

procesos nuevos, para que le ayuden a realizar su trabajo. En especial, es útil crear procesos cuando el trabajo a realizar se puede formular fácilmente en términos de varios procesos interactivos relacionados entre sí, pero independientes
En los sistemas interactivos, los usuarios pueden iniciar un programa escribiendo un **comando** o haciendo (doble) **clic** en un icono
En los sistemas de procesamiento por lotes que se encuentran en las mainframes grandes. Aquí los usuarios pueden enviar trabajos de procesamiento por lotes al sistema (posiblemente en forma remota). Cuando el sistema operativo decide que tiene los **recursos** para **ejecutar otro trabajo**, crea un proceso y ejecuta el siguiente trabajo de la cola de entrada.
En UNIX sólo hay una llamada al sistema para crear un proceso: **fork**. Esta llamada crea un **clon exacto del proceso que hizo la llamada.** Después de fork, los dos procesos (padre e hijo) tienen la **misma imagen de memoria**, las mismas **cadenas de entorno** y los mismos **archivos abiertos.** Eso es todo. Por lo general, el proceso hijo ejecuta después a **execve** o una llamada al sistema similar para **cambiar su imagen de memoria** y ejecutar un nuevo programa. Por el contrario, en Windows una sola llamada a una función de Win32 (**CreateProcess**) maneja la creación de procesos y carga el programa correcto en el nuevo proceso.

2.1.3 Terminación de procesos

1. Salida normal (voluntaria).
2. Salida por error (voluntaria).
3. Error fatal (involuntaria).
4. Eliminado por otro proceso (involuntaria)

La mayoría de los procesos terminan debido a que han **concluido su trabajo**. Cuando un compilador ha compilado el programa que recibe, ejecuta una llamada al sistema para indicar al sistema operativo que ha terminado. Esta llamada es exit en UNIX y ExitProcess en Windows. Los programas orientados a pantalla también admiten la terminación voluntaria.
La segunda razón de terminación es que el proceso descubra un **error**. Por ejemplo, si un usuario escribe el comando para ejecutar foo.c y no existe dicho archivo, el compilador simplemente termina.
La tercera razón de terminación es un **error fatal** producido por el **proceso**, a menudo debido a un error en el programa. Algunos ejemplos incluyen ejecutar una instrucción ilegal, hacer referencia a una parte de memoria no existente o la división entre cero.
La cuarta razón por la que un proceso podría terminar es que **ejecute una llamada al sistema** que indique al sistema operativo que **elimine otros procesos.** En UNIX esta llamada es kill. La función correspondiente en Win32 es TerminateProcess

2.1.4 Jerarquías de procesos

En algunos sistemas, cuando un proceso crea otro, **el proceso padre y el proceso hijo** continúan **asociados** en ciertas formas. El proceso hijo puede crear por sí mismo más procesos, formando una jerarquía de procesos.

En **UNIX**, un proceso y todos sus hijos y descendientes forman un **grupo de procesos**. Cuando un usuario envía una **señal desde el teclado**, ésta se envía a todos los procesos

del grupo asociado al teclado. Cada proceso puede **atrapar la señal, ignorarla o ser eliminado**.

Al iniciar la computadora, UNIX crea un proceso especial llamado **init**. Este proceso lee cuántas terminales hay y usa **fork** para crear un proceso por cada una. Luego, estos procesos esperan un inicio de sesión y, si es exitoso, ejecutan un **shell** que puede crear más procesos. Así, **todos los procesos del sistema forman un solo árbol**, con **init en la raíz**.

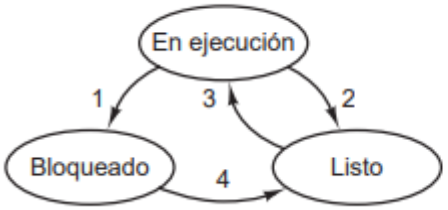
En **Windows**, no existe una jerarquía real de procesos: **todos los procesos son iguales**. Al crear un proceso, el padre recibe un **token (manejador)** para controlar al hijo, pero puede pasarlo a otros procesos, lo que **rompe la jerarquía**. En cambio, en UNIX **los procesos no pueden desheredar a sus hijos**.

Estados de un proceso

Cuando un proceso se bloquea, lo hace debido a que por lógica no puede continuar, comúnmente porque está esperando una entrada que todavía no está disponible. También es posible que un proceso, que esté listo en concepto y pueda ejecutarse, se detenga debido a que el sistema operativo ha decidido asignar la CPU a otro proceso por cierto tiempo

Los los tres estados en los que se puede encontrar un proceso son:

1. **En ejecución** (en realidad está usando la CPU en ese instante).
2. **Listo** (ejecutable; se detuvo temporalmente para dejar que se ejecute otro proceso).
3. **Bloqueado** (no puede ejecutarse sino hasta que ocurra cierto evento externo)



1. El proceso se bloquea para recibir entrada
2. El planificador selecciona otro proceso
3. El planificador selecciona este proceso
4. La entrada ya está disponible

Hay **cuatro transiciones** entre los **tres estados de un proceso**:

- La **transición 1** ocurre cuando un proceso no puede continuar y pasa a **bloqueado**, ya sea por una llamada al sistema o automáticamente (como en UNIX al leer sin datos disponibles).
- Las **transiciones 2 y 3** las maneja el **planificador**:
  la **2** cuando un proceso cede la CPU, y la **3** cuando vuelve a ejecutarse. La planificación busca **eficiencia y equidad**.
- La **transición 4** sucede cuando ocurre el **evento esperado**; el proceso pasa a **listo** o se ejecuta si la CPU está libre.

El **modelo de procesos** permite pensar el sistema como procesos que se bloquean y se desbloquean, con el **planificador** como nivel más bajo del sistema operativo.

Implementación de los procesos

Para implementar el modelo de procesos, el sistema operativo mantiene una tabla (un

arreglo de estructuras) llamada **tabla de procesos**, con sólo una **entrada por** cada **proceso** (algunos autores llaman a estas entradas bloques de control de procesos). Esta entrada contiene **información importante acerca del estado del proceso**, incluyendo su contador de programa, apuntador de pila, asignación de memoria, estado de sus archivos abiertos, información de contabilidad y planificación, etc.

Administración de procesos	Administración de memoria	Administración de archivos
Registros	Apuntador a la información del segmento de texto	Directorio raíz
Contador del programa	Apuntador a la información del segmento de datos	Directorio de trabajo
Palabra de estado del programa	Apuntador a la información del segmento de pila	Descripciones de archivos
Apuntador de la pila		ID de usuario
Estado del proceso		ID de grupo
Prioridad		
Parámetros de planificación		
ID del proceso		
Proceso padre		
Grupo de procesos		
Señales		
Tiempo de inicio del proceso		
Tiempo utilizado de la CPU		
Tiempo de la CPU utilizado por el hijo		
Hora de la siguiente alarma		

Figura 2-4. Algunos de los campos de una entrada típica en la tabla de procesos.

La **ilusión de varios procesos secuenciales** en una o varias CPU se mantiene mediante las **interrupciones**.
Cada clase de **E/S** tiene una ubicación asociada en memoria llamada **vector de interrupción**, que contiene la **dirección del procedimiento de servicio de interrupciones**.

1. El hardware mete el contador del programa a la pila, etc.
2. El hardware carga el nuevo contador de programa del vector de interrupciones.
3. Procedimiento en lenguaje ensamblador guarda los registros.
4. Procedimiento en lenguaje ensamblador establece la nueva pila.
5. El servicio de interrupciones de C se ejecuta (por lo general lee y guarda la entrada en el búfer).
6. El planificador decide qué proceso se va a ejecutar a continuación.
7. Procedimiento en C regresa al código de ensamblador.
8. Procedimiento en lenguaje ensamblador inicia el nuevo proceso actual.

Figura 2-5. Esqueleto de lo que hace el nivel más bajo del sistema operativo cuando ocurre una interrupción.

HILOS

En los sistemas operativos tradicionales, cada proceso tiene **un espacio** de **direcciones** y **un solo hilo de control**. De hecho, ésa es casi la definición de un proceso. Sin embargo, con frecuencia hay situaciones en las que es conveniente tener **varios hilos** de control en el mismo espacio de direcciones que se ejecuta en cuasi-paralelo
Uso de hilos

Los **hilos** son “miniprocesos” dentro de un proceso. Se usan porque muchas aplicaciones realizan **varias actividades a la vez**, algunas de las cuales pueden **bloquearse**. Al dividir una aplicación en varios hilos secuenciales que se ejecutan en **cuasi-paralelo**, el **modelo de programación se simplifica**.

A diferencia de los procesos, los **hilos comparten el mismo espacio de direcciones y los mismos datos**, lo cual es esencial para ciertas aplicaciones. Además, los hilos son **más**

ligeros, se **crean y destruyen más rápido** que los procesos, y permiten **traslapar cálculo y E/S**, mejorando el rendimiento cuando hay operaciones de disco.

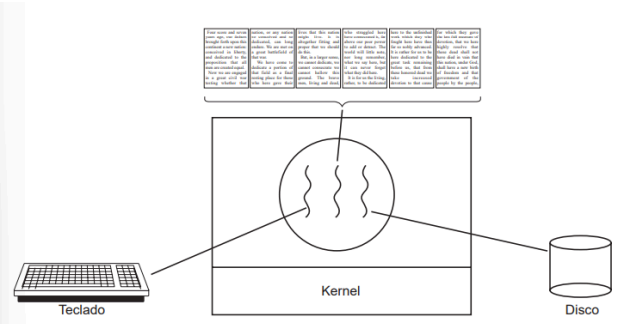


Figura 2-7. Un procesador de palabras con tres hilos.

En sistemas con **varias CPU**, los hilos permiten un paralelismo **real**.

Ejemplo:

- En un **procesador de palabras**, un hilo interactúa con el usuario, otro vuelve a dar formato en segundo plano y otro realiza **respaldos automáticos**, manteniendo buena respuesta.
- En un **servidor Web**, un hilo despachador recibe solicitudes y las asigna a **hilos trabajadores**, que comparten una **caché** y se bloquean si deben acceder al disco.

Los hilos permiten pensar los programas como **actividades paralelas simples**, algo que no sería posible usando procesos separados porque **no compartirían memoria**.

Un **servidor Web sin hilos** puede funcionar con **un solo hilo**, procesando una petición a la vez. Mientras espera al disco, la **CPU queda inactiva**, se procesan menos peticiones por segundo y el **rendimiento es bajo**.

Si no hay hilos, pero se quiere alto rendimiento, se puede usar **E/S sin bloqueo**. El servidor guarda el **estado de cada petición en una tabla** y reacciona a eventos (peticiones nuevas o respuestas del disco). Este diseño es una **máquina de estados finitos**, logra paralelismo, pero es **difícil de programar**.

Los **hilos** combinan lo mejor: **paralelismo** con **llamadas al sistema con bloqueo**, lo que **simplifica la programación** y **mejora el rendimiento**.

Comparación:

- **Hilos**: paralelismo + llamadas con bloqueo
- **Un solo hilo**: sin paralelismo + llamadas con bloqueo
- **Máquina de estados finitos**: paralelismo + llamadas sin bloqueo e interrupciones

Los hilos también son útiles para **procesar grandes volúmenes de datos**, separando **entrada, procesamiento y salida** en distintos hilos, permitiendo que todo ocurra al mismo tiempo sin dejar la CPU inactiva.

Modelo clásico de hilos

El **modelo clásico de hilos** separa dos conceptos del proceso: **agrupamiento de recursos** y **ejecución**.
El **proceso** agrupa recursos (espacio de direcciones, archivos abiertos, señales, procesos hijos, etc.), mientras que el **hilo** es la unidad de **ejecución**, con su **contador de programa, registros y pila**.

Los **hilos permiten varias ejecuciones dentro del mismo proceso**, compartiendo el **mismo espacio de direcciones y recursos**. Por eso se los llama **procesos ligeros**. A diferencia de varios procesos, los hilos **cooperan** y no tienen protección entre ellos.

Un proceso puede tener **uno o varios hilos**:

- Varios procesos → cada uno con su propio espacio de direcciones.
- Un proceso multihilo → varios hilos **comparten memoria**.

En una sola CPU, los hilos **se turnan** para ejecutarse, dando la **ilusión de paralelismo**. Cada hilo tiene su **propia pila**, necesaria porque cada uno tiene un historial de ejecución distinto.

Los hilos pueden estar en los mismos estados que los procesos: **ejecución, bloqueado, listo o terminado**.
Se crean con llamadas como **thread_create**, terminan con **thread_exit**, pueden esperar a otros con **thread_join** y ceder la CPU con **thread_yield**.

Aunque son muy útiles, los hilos **complican la programación**:

- Problemas con **fork** en procesos con varios hilos.
- Conflictos por **datos compartidos** (archivos, memoria, recursos).

Por eso, los programas multihilo requieren un diseño **cuidadoso** para funcionar correctamente.

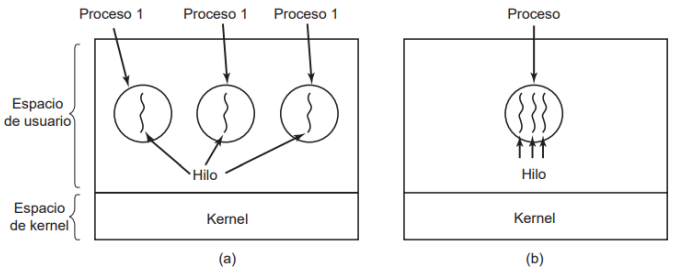


Figura 2-11. (a) Tres procesos, cada uno con un hilo. (b) Un proceso con tres hilos.

Elementos por proceso	Elementos por hilo
Espacio de direcciones	Contador de programa
Variables globales	Registros
Archivos abiertos	Pila
Procesos hijos	Estado
Alarmas pendientes	
Señales y manejadores de señales	
Información contable	

Al igual que un proceso tradicional (es decir, un proceso con sólo un hilo), un hilo puede estar en uno de varios **estados**: en ejecución, bloqueado, listo o terminado.
Implementación de hilos en el espacio de usuario

Hay **dos formas principales de implementar hilos**: en **espacio de usuario** o en el **kernel** (también existe un modelo híbrido).

- En los **hilos de nivel usuario**, el kernel **no sabe que existen** y ve sólo procesos con un hilo. Los hilos se implementan mediante una **biblioteca** y un **sistema en tiempo de ejecución**, que mantiene una **tabla de hilos** por proceso.
  La **ventaja principal** es el **alto rendimiento**: la creación, planificación y conmutación de hilos es **muy rápida**, sin traps al kernel. Además, cada proceso puede usar su **propio algoritmo de planificación** y escalan mejor cuando hay muchos hilos.

Las **desventajas** son importantes:

- Una **llamada al sistema con bloqueo** bloquea **todo el proceso**, no sólo el hilo.
- Los **fallos de página** también bloquean a todos los hilos.
- No hay **interrupciones de reloj**, por lo que un hilo puede quedarse con la CPU si no la cede voluntariamente.
- Para evitar bloqueos se necesitan **llamadas sin bloqueo**, `select` y **envolturas**, lo que vuelve el código **complejo e ineficiente**.

En resumen:
Los **hilos de nivel usuario** son **rápidos**, pero **difíciles de usar correctamente**, y funcionan mal en aplicaciones que hacen **muchas llamadas al sistema**, que son justamente donde más se desean los hilos.

Implementación de hilos en el kernel

- En los **hilos de nivel kernel**, el **kernel conoce y administra los hilos**. Mantiene una **tabla global de hilos**, con su estado, registros y demás información. Crear o destruir hilos requiere **llamadas al sistema**, lo que implica **mayor costo** que en hilos de nivel usuario.

Ventajas principales:

- Si un hilo se **bloquea** o ocurre un **fallo de página**, el kernel puede ejecutar **otro hilo del mismo proceso o de otro proceso**.
- No se necesitan llamadas al sistema **sin bloqueo** ni envolturas.

- Funcionan bien en aplicaciones con **muchas llamadas al sistema**.

Desventaja principal:

- **Alta sobrecarga**: crear, destruir y manejar hilos es más lento porque todo pasa por el kernel. Para reducir esto, algunos sistemas **reciclan hilos**.

Los hilos de kernel **no resuelven todos los problemas**, como:

- Qué hacer con los hilos al usar **fork**.
- Cómo manejar las **señales**, que se envían a procesos y no a hilos.

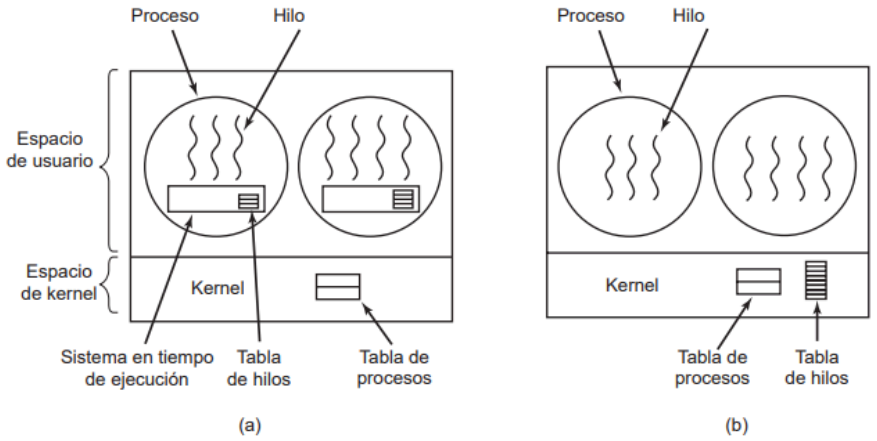


Figura 2-16. (a) Un paquete de hilos de nivel usuario. (b) Un paquete de hilos administrado por el kernel.

Implementaciones híbridas:

- Combinan **hilos de kernel** con **hilos de usuario**.
  El kernel planifica los hilos de kernel, y **cada hilo de kernel puede ejecutar varios hilos de usuario multiplexados**.
  Este modelo ofrece **máxima flexibilidad**, combinando rendimiento y control.

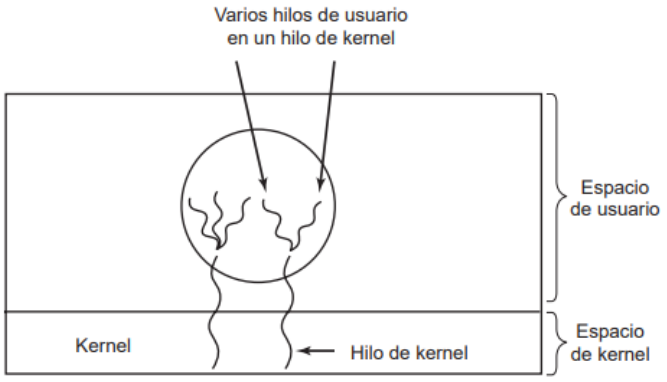


Figura 2-17. Multiplexaje de hilos del nivel usuario sobre hilos del nivel kernel.

Activaciones del planificador

Las **activaciones del planificador** buscan combinar lo mejor de los **hilos de kernel** y los **hilos de nivel usuario**: mantener el **paralelismo real y el manejo correcto de bloqueos**, pero con **menor sobrecarga y mayor eficiencia**.

La idea central es que el **kernel asigna procesadores virtuales** a cada proceso y el **sistema en tiempo de ejecución (en espacio de usuario)** decide qué hilos ejecutar en ellos.
Cuando un hilo se **bloquea** (llamada al sistema o fallo de página), el kernel **notifica al sistema en tiempo de ejecución** mediante una **llamada ascendente (upcall)**, permitiéndole **replanificar otro hilo** sin bloquear todo el proceso.

Cuando el evento que causó el bloqueo se resuelve, el kernel vuelve a avisar con otra **upcall**, y el sistema decide **reanudar el hilo o dejarlo listo**.
Esto evita transiciones innecesarias entre **usuario y kernel**, mejorando el rendimiento.

La **principal crítica** es que las **llamadas ascendentes rompen el modelo clásico por capas**, ya que el kernel llama a código de nivel superior.

Hilos emergentes

Las **activaciones del planificador** buscan combinar lo mejor de los **hilos de kernel** y los **hilos de nivel usuario**:
mantener el **paralelismo real y el manejo correcto de bloqueos**, pero con **menor sobrecarga y mayor eficiencia**.

La idea central es que el **kernel asigna procesadores virtuales** a cada proceso y el **sistema en tiempo de ejecución (en espacio de usuario)** decide qué hilos ejecutar en ellos.
Cuando un hilo se **bloquea** (llamada al sistema o fallo de página), el kernel **notifica al sistema en tiempo de ejecución** mediante una **llamada ascendente (upcall)**, permitiéndole **replanificar otro hilo** sin bloquear todo el proceso.

Cuando el evento que causó el bloqueo se resuelve, el kernel vuelve a avisar con otra **upcall**, y el sistema decide **reanudar el hilo o dejarlo listo**.
Esto evita transiciones innecesarias entre **usuario y kernel**, mejorando el rendimiento.

La **principal crítica** es que las **llamadas ascendentes rompen el modelo clásico por capas**, ya que el kernel llama a código de nivel superior.

Los **hilos emergentes** son otra técnica usada en sistemas distribuidos:
La llegada de un mensaje **crea automáticamente un nuevo hilo** para procesarlo. Como el hilo empieza desde cero, se **reduce la latencia**.
Pueden ejecutarse en **kernel** (más rápido y potente, pero más riesgoso) o en **usuario** (más seguro).

Los **hilos emergentes** son otra técnica usada en sistemas distribuidos:
La llegada de un mensaje **crea automáticamente un nuevo hilo** para procesarlo. Como el hilo empieza desde cero, se **reduce la latencia**.

Pueden ejecutarse en **kernel** (más rápido y potente, pero más riesgoso) o en **usuario** (más seguro).

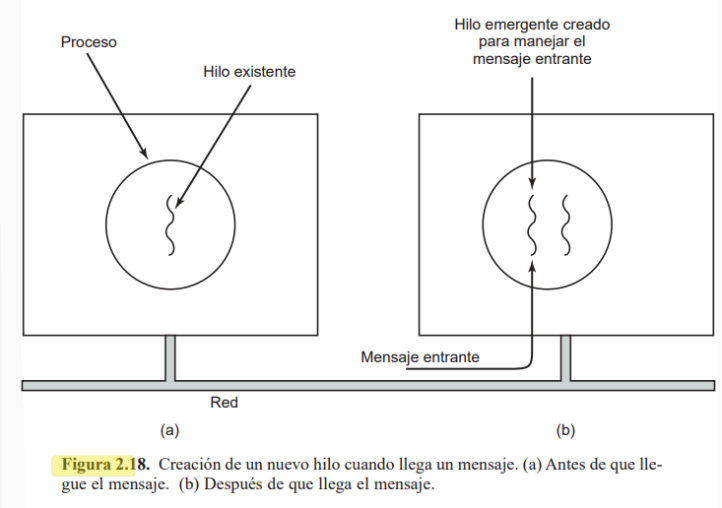
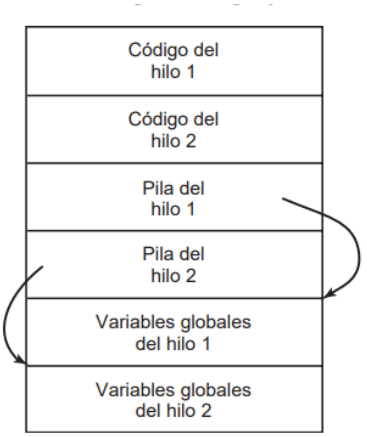


Figura 2.18. Creación de un nuevo hilo cuando llega un mensaje. (a) Antes de que llegue el mensaje. (b) Después de que llega el mensaje.

Convertir un programa de **un solo hilo a multihilo es difícil**.
Los principales problemas son:



- **Variables globales** (como `errno`) generan **conflictos entre hilos**.
- Muchas **bibliotecas no son reentrantes** y fallan con varios hilos.
- La **gestión de señales** se vuelve ambigua y compleja.
- La **administración de múltiples pilas** no está bien soportada si el kernel no conoce los hilos.

Conclusión:
Agregar hilos **no es automático**; requiere **rediseñar bibliotecas, semántica de llamadas al sistema y manejo de recursos**, manteniendo compatibilidad con programas existentes.

Comunicación entre procesos

Con frecuencia, los **procesos necesitan comunicarse con otros procesos**. Un ejemplo típico es una **canalización del shell**, donde la salida de un proceso se pasa al siguiente.
Esta necesidad se denomina **comunicación entre procesos (IPC)** y se busca que sea **bien estructurada**, sin usar interrupciones.

Existen **tres cuestiones fundamentales** en la IPC:

1. **Cómo un proceso pasa información a otro**.

2. **Cómo evitar que los procesos se interpongan entre sí** al usar recursos compartidos.
3. **Cómo asegurar la secuencia correcta** cuando hay dependencias (por ejemplo, que un proceso espere a que otro produzca datos).

Estas tres cuestiones se analizarán en las secciones siguientes.

Aunque se estudian en el contexto de los **procesos**, **las dos últimas también se aplican a los hilos**.
La primera (paso de información) es más sencilla en hilos porque **comparten el espacio de direcciones**, pero **la exclusión mutua y la sincronización** presentan los mismos problemas y requieren las mismas soluciones.

Condiciones de carrera

En algunos sistemas operativos, los procesos que cooperan pueden **compartir almacenamiento**, ya sea en **memoria principal** o en **archivos compartidos**.
La ubicación no cambia la naturaleza del problema.

Ejemplo: **spooler de impresión**.

- Un proceso coloca el nombre del archivo a imprimir en un directorio compartido.
- Un demonio de impresión revisa ese directorio y procesa los archivos.

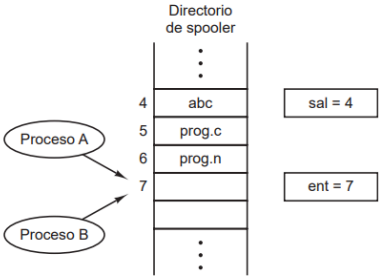


Figura 2-21. Dos procesos desean acceder a la memoria compartida al mismo tiempo.

Supóngase que:

- `sal` apunta al **siguiente archivo a imprimir**.
- `ent` apunta a la **siguiente ranura libre** del directorio.

Si **dos procesos acceden simultáneamente** a `ent`, ambos pueden creer que la misma ranura está libre y **sobrescribir información**, provocando que **uno de los trabajos se pierda**, sin que el sistema detecte el error.

Este tipo de situación, donde:

- dos o más procesos **leen o escriben datos compartidos**, y
- el resultado **depende del orden y el momento de ejecución**,

se denomina **condición de carrera**.

Las condiciones de carrera son **difíciles de depurar** porque la mayoría de las veces el programa funciona bien, pero **ocasionalmente falla de manera inexplicable**.

Regiones críticas

Para evitar condiciones de carrera, es necesario impedir que **más de un proceso acceda al mismo tiempo** a los datos compartidos.
Esto se logra mediante **exclusión mutua**.

La **región crítica** (o **sección crítica**) es la parte del programa donde un proceso:

- accede a **memoria compartida**,
- archivos compartidos,
- o cualquier recurso que pueda producir condiciones de carrera.

Si se garantiza que **nunca haya dos procesos simultáneamente en sus regiones críticas**, se evitan las condiciones de carrera.

Sin embargo, una solución correcta debe cumplir **cuatro condiciones**:

1. **No puede haber dos procesos al mismo tiempo en sus regiones críticas.**
2. **No deben hacerse suposiciones sobre velocidades ni cantidad de CPUs.**
3. **Un proceso fuera de su región crítica no debe bloquear a otros.**
4. **Ningún proceso debe esperar indefinidamente para entrar a su región crítica.**

El comportamiento deseado es:

- Un proceso entra a su región crítica.
- Otro proceso que intenta entrar **debe esperar**.
- Cuando el primero sale, el segundo **entra inmediatamente**.

De esta forma, los procesos **cooperan correctamente**, sin interferencias ni bloqueos injustos.

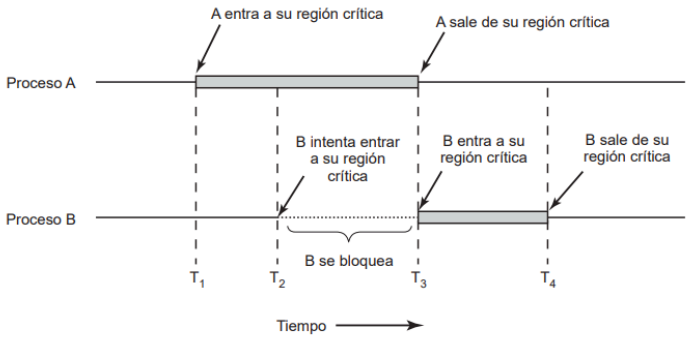


Figura 2-22. Exclusión mutua mediante el uso de regiones críticas.

Exclusión mutua con espera ocupada

En esta sección se analizan **propuestas para lograr exclusión mutua**, de modo que **mientras un proceso esté en su región crítica**, ningún otro proceso pueda entrar y provocar **condiciones de carrera**.

Las soluciones presentadas utilizan **espera ocupada** (busy waiting), es decir, procesos que **esperan activamente** hasta poder entrar.

Deshabilitando interrupciones

En un sistema con **un solo procesador**, la solución más simple es que un proceso **deshabilite las interrupciones** al entrar a su región crítica y las **rehabilite al salir**.

- Sin interrupciones, **no hay conmutación de procesos**, porque la CPU sólo cambia de proceso por una interrupción.
- Así, el proceso puede **examinar y actualizar memoria compartida sin interferencias**.

Problemas:

- **No es seguro** permitir que procesos de usuario deshabiliten interrupciones: si no las reactivan, el sistema puede quedar inutilizado.
- En **sistemas multiprocesador**, deshabilitar interrupciones **sólo afecta a una CPU**; las demás siguen ejecutándose y pueden acceder a la memoria compartida.
- Por eso, **no es una solución general válida** para procesos de usuario.

Conclusión:

- Deshabilitar interrupciones es **útil dentro del kernel**, para proteger estructuras internas.
- **No es apropiado como mecanismo general de exclusión mutua**, y su utilidad disminuye aún más con los **sistemas multinúcleo**.

Variables de candado

Se propone una solución puramente de **software** usando una **variable compartida llamada candado**:

- `candado = 0` → nadie está en la región crítica
- `candado = 1` → algún proceso está en la región crítica

Un proceso:

1. Verifica el candado.
2. Si es 0, lo pone en 1 y entra.
3. Si es 1, espera hasta que vuelva a 0.

Problema:

Esta solución tiene **exactamente el mismo error que el spooler**:

- Dos procesos pueden leer el candado como 0 **al mismo tiempo**.
- Ambos lo fijan en 1 y **entran simultáneamente** a la región crítica.

Verificar el candado más de una vez **no resuelve el problema**, porque la condición de carrera puede ocurrir entre verificaciones.

Conclusión:

- **No garantiza exclusión mutua**.
- No es una solución válida.

Alternancia estricta

Se introduce una variable compartida `turno`, que indica **qué proceso puede entrar** a la región crítica.

- Cada proceso espera hasta que `turno` tenga su valor.
- Al salir de la región crítica, cambia `turno` para ceder el acceso al otro proceso.
- El proceso que no tiene el turno **espera activamente** (espera ocupada).

La **espera ocupada** consiste en evaluar continuamente una variable hasta que cambie.
Un candado que usa este método se llama **candado de giro**.

Problema principal:

Aunque **evita condiciones de carrera**, **viola la condición 3** de las regiones críticas:

> *Un proceso fuera de su región crítica no debe bloquear a otros.*

Si un proceso es lento o está en su región no crítica, el otro proceso puede quedar **bloqueado innecesariamente**, aunque nadie esté usando la región crítica.

Consecuencia:

- Los procesos deben **alternarse estrictamente**, incluso cuando no es necesario.
- Un proceso no puede entrar dos veces seguidas a su región crítica.
- Esto provoca **ineficiencia y bloqueos injustificados**.

Conclusión:

- Aunque es correcta desde el punto de vista de exclusión mutua,
  no es una solución práctica ni aceptable, porque **bloquea procesos sin motivo**.

Solución de Peterson

La **solución de Peterson** es un **algoritmo de software** para lograr **exclusión mutua sin alternancia estricta**.

Fue propuesta por **G. L. Peterson (1981)** y reemplazó a la solución más compleja de **Dekker**.

Utiliza:

- una variable **turno**, y
- un arreglo **interesado[]** que indica si un proceso quiere entrar a la región crítica.

Antes de entrar a la **región crítica**, cada proceso:

1. Indica que está interesado.
2. Establece el turno.
3. Espera mientras el otro proceso esté interesado **y** sea su turno.

Al salir de la región crítica, el proceso:

- marca que **ya no está interesado**.

Propiedades importantes:

- Garantiza **exclusión mutua**.
- No requiere **alternancia estricta**.
- Funciona incluso si ambos procesos intentan entrar **casi simultáneamente**.
- Utiliza **espera ocupada**.

Cuando ambos procesos quieren entrar:

- El último en escribir `turno` cede el acceso al otro.
- Uno entra a la región crítica y el otro espera.

La instrucción TSL

La instrucción **TSL (Test and Set Lock)** es una **instrucción de hardware** que:

- lee el valor de una variable compartida (candado),
- la guarda en un registro,
- y luego **la fija en un valor distinto de cero**,
  todo de manera **indivisible (atómica)**.

Durante la ejecución de TSL:

- se **bloquea el bus de memoria**,
- ningún otro procesador puede acceder a esa variable.

Esto es **muy distinto** de deshabilitar interrupciones:

- deshabilitar interrupciones **no evita** que otros procesadores accedan a la memoria.

Uso de TSL:

- Si `candado = 0`, el proceso lo fija en 1 y entra a la región crítica.
- Si `candado ≠ 0`, el proceso **espera ocupadamente** hasta que se libere.

Al salir de la región crítica:

- el proceso vuelve a poner `candado = 0`.

Características:

- Solución **simple y directa**.
- Requiere **espera ocupada**.
- La exclusión mutua depende de que los procesos **llamen correctamente** a entrar y salir de la región crítica.

La instrucción XCHG

La instrucción **XCHG** intercambia de forma **atómica**:

- el contenido de un registro,
- con una palabra de memoria (el candado).

Funciona de manera equivalente a **TSL**:

- si el valor previo del candado era 0, el proceso entra,
- si no, sigue esperando.

Nota importante:

- Las CPUs **Intel x86** utilizan **XCHG** para sincronización de bajo nivel.

2.3.4 Dormir y despertar

Las soluciones anteriores (**Peterson, TSL, XCHG**) son correctas, pero **todas usan espera ocupada**, lo cual:

- desperdicia CPU,
- puede causar efectos inesperados.

Inversión de prioridades

Si un proceso de **alta prioridad** entra en espera ocupada mientras uno de **baja prioridad** está en la región crítica:

- el proceso de baja prioridad **nunca se ejecuta**,
- el de alta prioridad **espera indefinidamente**.

Esto se conoce como **inversión de prioridades**.

Primitivas sleep y wakeup

Para evitar la espera ocupada, se introducen primitivas que **bloquean procesos**:

- **sleep**: el proceso se bloquea (queda inactivo).
- **wakeup**: despierta a un proceso bloqueado.

Estas primitivas permiten que un proceso **no consuma CPU** mientras espera.

El problema del productor–consumidor

Dos procesos comparten un **búfer limitado**:

- el **productor** inserta elementos,
- el **consumidor** los retira.

Reglas:

- Si el búfer está **lleno**, el productor se duerme.
- Si el búfer está **vacío**, el consumidor se duerme.
- Cada uno despierta al otro cuando corresponde.

Para controlar el búfer se usa una variable compartida **cuenta**.

Condición de carrera en sleep/wakeup

El acceso a **cuenta** no está protegido, lo que produce una **condición de carrera fatal**.

Puede ocurrir que:

- el productor envíe un **wakeup** cuando el consumidor **todavía no está dormido**,
- la señal se **pierda**,
- luego ambos procesos se duerman **para siempre**.

Problema central:

> Una señal de despertar enviada a un proceso que aún no duerme **se pierde**.

Bit de espera de despertar

Una solución parcial es agregar un **bit de espera de despertar**:

- si llega un wakeup y el proceso no está dormido, se guarda la señal.
- cuando el proceso intenta dormir, consume ese bit y **no se duerme**.

Limitación:

- Funciona en casos simples.
- **No escala** bien con múltiples procesos.
- El problema fundamental **sigue existiendo**.

2.3.5 Semáforos

En 1965, **E. W. Dijkstra** propuso una solución general al problema de la **pérdida de señales de despertar** introduciendo una nueva variable: el **semáforo**.

Un **semáforo** es una **variable entera** que:

- vale **0** si no hay señales de despertar almacenadas,
- vale un **número positivo** si hay una o más señales pendientes.

Operaciones sobre semáforos

Dijkstra definió dos operaciones:

- **down** (generalización de *sleep*)
- **up** (generalización de *wakeup*)

Operación down

- Si el valor del semáforo es mayor que 0, lo decrementa y continúa.
- Si el valor es 0, el proceso se **bloquea**.
- La comprobación, modificación y bloqueo se realizan como una **acción atómica indivisible**.

Operación up

- Incrementa el valor del semáforo.
- Si hay procesos bloqueados, **despierta uno** de ellos.
- También es una operación **atómica**.
- Ningún proceso se bloquea al ejecutar up.

La **atomicidad** es esencial para evitar **condiciones de carrera**.

Semáforos y productor–consumidor

Los semáforos resuelven el problema del productor–consumidor **sin pérdida de señales**.

Se utilizan **tres semáforos**:

- **llenas**: cuenta ranuras ocupadas
- **vacías**: cuenta ranuras libres
- **mutex**: asegura exclusión mutua

Valores iniciales:

- llenas = 0
- vacías = N
- mutex = 1

El semáforo **mutex** es un **semáforo binario** y se usa para **exclusión mutua**.
Los semáforos **llenas** y **vacías** se usan para **sincronización**.

Dos usos distintos del semáforo

1. **Exclusión mutua** → mutex
2. **Sincronización de eventos** → llenas y vacías

Semáforos e interrupciones

En sistemas con E/S:

- Cada dispositivo puede tener un **semáforo asociado**.
- El proceso realiza **down** y se bloquea.
- El manejador de interrupciones realiza **up** y despierta al proceso.

Así, las interrupciones quedan **ocultas** mediante semáforos.

2.3.6 Mutexes

Un **mutex** es una versión **simplificada** del semáforo:

- sólo sirve para **exclusión mutua**,
- **no cuenta señales**.

Un mutex puede estar:

- **abierto** (0)
- **cerrado** (≠ 0)

Operaciones

- **mutex_lock**: adquiere el mutex o se bloquea.
- **mutex_unlock**: libera el mutex y despierta a un hilo.

Los mutexes son:

- simples,
- eficientes,
- ideales para **hilos de nivel usuario**.

Diferencia clave con espera ocupada

En hilos de usuario:

- no hay reloj que fuerce el cambio de hilo
- la espera ocupada puede bloquear el sistema.

Por eso, **mutex_lock** llama a **thread_yield**:

- cede la CPU a otro hilo,
- evita espera ocupada.

Todo ocurre **en espacio de usuario**, sin llamadas al kernel.

Mutexes y memoria compartida

Para que mutexes, semáforos o Peterson funcionen entre procesos:

- debe existir **memoria compartida**.

Opciones:

1. estructuras en el **kernel** accesibles por llamadas al sistema,
2. **segmentos de memoria compartida** entre procesos,
3. archivos compartidos (peor caso).

Aunque procesos compartan memoria, **no son hilos**:

- los procesos siguen teniendo recursos separados.

Mutexes en Pthreads

Pthreads ofrece mutexes con las siguientes operaciones:

- `pthread_mutex_init`
- `pthread_mutex_destroy`
- `pthread_mutex_lock`
- `pthread_mutex_trylock`
- `pthread_mutex_unlock`

Los mutexes **no son obligatorios**:

- el programador debe usarlos correctamente.

Variables de condición (Pthreads)

Las **variables de condición** permiten que un hilo:

- se bloquee hasta que **una condición se cumpla**.

Operaciones principales:

- **pthread_cond_wait**
- **pthread_cond_signal**
- **pthread_cond_broadcast**

Características clave:

- siempre se usan **junto con mutexes**,
- `pthread_cond_wait` libera y recupera el mutex **en forma atómica**,
- **no tienen memoria**: si no hay hilos esperando, la señal se pierde.

2.4 PLANIFICACIÓN

Cuando una computadora se multiprograma, existen **varios procesos o hilos en estado listo** que compiten por la CPU.
Si hay **una sola CPU**, es necesario decidir **cuál se ejecuta a continuación**.

- El **planificador de procesos** es la parte del sistema operativo que toma esa decisión.

- El **algoritmo de planificación** es el criterio que utiliza el planificador para decidir.

Muchas de las cuestiones de planificación aplican tanto a **procesos como a hilos**.
Cuando el kernel administra hilos, la planificación suele hacerse **por hilo**, sin importar demasiado a qué proceso pertenece.

En esta sección se analizan primero los conceptos generales y luego los **algoritmos concretos**, distinguiendo distintos tipos de sistemas.

Un buen planificador **impacta directamente** en:

- rendimiento del sistema
- tiempo de respuesta
- satisfacción del usuario

Computadoras personales

En las PCs modernas:

- Normalmente hay **un solo proceso activo**
- La CPU **no suele ser un recurso escaso**
- La planificación **no es crítica** en la mayoría de los casos

Existen excepciones (video, multimedia, cómputo intensivo), pero **no son la regla**.

Servidores

En **servidores en red**, la situación cambia:

- Muchos procesos compiten constantemente por la CPU
- La planificación vuelve a ser **muy importante**
- Se priorizan procesos que atienden usuarios frente a tareas en segundo plano

Costo de la conmutación de procesos

Cambiar de proceso **no es gratis**:

- cambio de modo usuario → kernel
- guardado de registros y estado
- cambio de mapas de memoria
- posible invalidación de caché

Demasiadas conmutaciones degradan el rendimiento, por lo que el planificador debe ser cuidadoso.

Comportamiento de un proceso

Los procesos **alternan** entre:

- **ráfagas de CPU**
- **esperas por E/S**

Esto da lugar a dos tipos de procesos:

Procesos limitados a cálculos (CPU-bound)

- Ráfagas de CPU **largas**
- Esperas de E/S **poco frecuentes**

Procesos limitados a E/S (I/O-bound)

- Ráfagas de CPU **cortas**
- Esperas de E/S **frecuentes**

Lo importante es la **duración de la ráfaga de CPU**, no la de la E/S.

A medida que las CPUs son más rápidas que los discos:

- los procesos tienden a ser **más limitados a E/S**
- planificarlos bien es cada vez más importante

Cuándo planificar procesos

El planificador debe tomar decisiones en varias situaciones:

1. **Creación de un proceso**
    - decidir si se ejecuta el padre o el hijo
2. **Terminación de un proceso**
    - elegir otro proceso listo o uno inactivo
3. **Bloqueo de un proceso**
    - por E/S, semáforos, etc.
4. **Interrupciones de E/S**
    - decidir si ejecutar el proceso desbloqueado o continuar con el actual
5. **Interrupciones de reloj**
    - permiten o no la apropiación

Planificación no apropiativa

- El proceso se ejecuta **hasta que se bloquea o libera la CPU**
- No se lo interrumpe por tiempo
- Más simple, menos sobrecarga

Planificación apropiativa

- Cada proceso tiene un **tiempo máximo**
- Si se agota, se suspende
- Requiere interrupciones de reloj

Categorías de los algoritmos de planificación

Según el tipo de sistema:

1. Procesamiento por lotes

- No hay usuarios esperando
- Se aceptan algoritmos no apropiativos
- Se busca eficiencia global

2. Sistemas interactivos

- Usuarios esperan respuestas rápidas
- La apropiación es **esencial**
- Incluye servidores

3. Sistemas de tiempo real

- Existen **tiempos límite**
- Importa cumplir plazos y la predictibilidad
- No se ejecutan programas arbitrarios

Metas de los algoritmos de planificación

En todos los sistemas

- **Equidad**: procesos similares deben recibir servicio similar
- **Aplicación de políticas**: cumplir prioridades del sistema
- **Balance**: mantener CPU y E/S ocupados

Procesamiento por lotes

- **Rendimiento**: trabajos por hora
- **Tiempo de retorno**: desde envío hasta terminación
- **Utilización de la CPU** (métrica secundaria)

Sistemas interactivos

- **Tiempo de respuesta**
- **Proporcionalidad** (cumplir expectativas del usuario)

Sistemas de tiempo real

- **Cumplir plazos**
- **Predictibilidad**

2.4.2 Planificación en sistemas de procesamiento por lotes

Primero en entrar, primero en ser atendido (FCFS)

- Algoritmo **no apropiativo**
- Se ejecutan los procesos **en orden de llegada**
- Implementación simple: una cola

Ventaja

- Fácil de entender
- Equitativo en sentido formal

Desventaja

- Perjudica procesos limitados a E/S
- Un proceso largo puede retrasar muchos cortos

Trabajo más corto primero (SJF)

- Algoritmo **no apropiativo**
- Supone conocer el tiempo de ejecución
- Ejecuta primero el trabajo más corto

Resultado

- Minimiza el **tiempo promedio de retorno**

- Es óptimo **solo si todos los trabajos llegan juntos**

Menor tiempo restante a continuación (SRTN)

- Versión **apropiativa** de SJF
- Siempre ejecuta el proceso con **menor tiempo restante**
- Un proceso nuevo puede desplazar al actual

Ventaja

- Excelente tiempo de respuesta para trabajos cortos
- Penaliza menos a procesos interactivos



Figura 2-41. Planificación de turno circular. (a) La lista de procesos ejecutables. (b) La lista de procesos ejecutables una vez que *B* utiliza su quántum.

:

2.4.3 Planificación en sistemas interactivos

Los **sistemas interactivos** (PCs, servidores, sistemas multiusuario) necesitan **tiempos de respuesta rápidos**. El objetivo principal no es maximizar el rendimiento total, sino **responder rápido a acciones cortas del usuario**.

Planificación por turno circular (Round-Robin)

Es uno de los algoritmos **más usados, simples y equitativos**.

- A cada proceso se le asigna un **intervalo de tiempo fijo**, llamado **quántum**.
- El proceso se ejecuta **hasta que:
  - termina,
  - se bloquea,
  - o se agota su quántum**.
- Si se agota el quántum, el proceso se **interrumpe** y se coloca **al final de la cola** de procesos listos.

Idea clave: todos los procesos obtienen CPU por turnos.

El problema del tamaño del quántum

- **Quántum muy corto**
  → muchas conmutaciones de proceso
  → mucho tiempo perdido en sobrecarga administrativa
  → baja eficiencia de la CPU

- **Quántum muy largo**
  → mala respuesta a procesos interactivos cortos
  → un proceso puede esperar varios segundos antes de ejecutarse

Conclusión:
Un quántum **intermedio (20–50 ms)** suele ser una solución razonable.

Planificación por prioridad

No todos los procesos tienen la misma importancia.

- A cada proceso se le asigna una **prioridad**.
- **Siempre se ejecuta el proceso listo con mayor prioridad**.

Ejemplos:

- Un proceso que reproduce video en tiempo real tiene más prioridad que uno que envía correos en segundo plano.
- En sistemas multiusuario, distintos usuarios pueden tener prioridades distintas.

Evitar monopolios de CPU

Para evitar que un proceso de alta prioridad se ejecute indefinidamente:

- Se puede **reducir su prioridad con el tiempo**, o
- Limitar el tiempo máximo que puede ejecutarse (quántum).

Prioridades estáticas y dinámicas

- **Estáticas**: asignadas desde el inicio (ej.: jerarquías, costos, niveles).
- **Dinámicas**: cambian según el comportamiento del proceso.

Ejemplo importante:

- Los procesos **limitados a E/S** deben recibir la CPU rápido para iniciar su próxima operación.
- Una política simple es asignar prioridad proporcional a **1/f**, donde *f* es la fracción del quántum utilizado.

Múltiples colas de prioridad

Se agrupan procesos en **clases de prioridad**.

- El planificador:
  - ejecuta primero todos los procesos de la **clase más alta**,
  - usando **turno circular dentro de cada clase**.

- Las clases de menor prioridad **no se ejecutan** mientras haya procesos en clases superiores.

Problema:
Si no se reajustan prioridades, los procesos de baja prioridad pueden **morir de hambre** (starvation).

Históricamente:

- Sistemas como **CTSS** y **XDS 940** usaron múltiples colas para:
    - favorecer procesos interactivos,
    - reducir el costo de conmutación,
    - penalizar procesos muy largos.

El proceso más corto a continuación

Busca aplicar la idea de **trabajo más corto primero** a sistemas interactivos.

- Cada comando interactivo se considera un "trabajo".
- Se ejecuta el proceso con **menor tiempo estimado de ejecución**.

Como el tiempo real no se conoce:

- Se estima usando **promedios ponderados** de ejecuciones anteriores.
- Técnica conocida como **envejecimiento (aging)**.

Muy importante:

- Se usa el comportamiento pasado para predecir el futuro.
- Favorece comandos cortos y mejora el tiempo de respuesta.

Planificación garantizada

El sistema hace una **promesa explícita**:

> "Si hay n procesos activos, cada uno recibirá aproximadamente 1/n del tiempo de CPU."

- El sistema mide:
    - cuánto CPU recibió cada proceso,
    - cuánto debería haber recibido.
- Se ejecuta el proceso que **menos ha recibido en proporción a lo que le corresponde**.

Planificación por sorteo (lotería)

Cada proceso recibe **boletos de lotería**.

- En cada decisión de planificación:
    - se sortea un boleto,
    - el proceso ganador obtiene CPU.

Características clave:

- Más boletos → más probabilidad de ejecutarse.
- Fácil de entender y de implementar.
- Muy flexible.

Ventajas importantes:

- Alta capacidad de respuesta.
- Procesos pueden **transferir boletos** (cliente-servidor).
- Ideal para repartir CPU en proporciones exactas (ej.: video).

Planificación por partes equitativas

La planificación se hace **por usuario**, no por proceso.

- Cada usuario recibe una fracción de la CPU.
- Sus procesos se planifican dentro de ese límite.

Ejemplo:

- Usuario A tiene 4 procesos.
- Usuario B tiene 1 proceso.
- Ambos reciben 50% de la CPU.

Evita que un usuario "gane" CPU sólo por crear más procesos.

2.4.4 Planificación en sistemas de tiempo real

En los **sistemas de tiempo real**, **el tiempo es crítico**.

- Obtener la respuesta correcta **fuera de tiempo** puede ser tan grave como no obtenerla.
- Ejemplos: reproductores de audio, control industrial, monitoreo médico, robots.

Tipos de tiempo real

- **Tiempo real duro:** los tiempos límite **deben cumplirse siempre**.
- **Tiempo real suave:** fallar ocasionalmente es tolerable, pero degrada la calidad.

Eventos periódicos y aperiódicos

- **Periódicos**: ocurren a intervalos regulares.
- **Aperiódicos**: ocurren de forma impredecible.

Un sistema es **planificable** si la suma de las cargas cumple:

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

Donde:

- Ci: tiempo de CPU requerido por evento
- Pi: período del evento

Si no se cumple, **no es posible garantizar los tiempos límite**.

Planificación estática y dinámica

- **Estática**:
    - decisiones tomadas antes de ejecutar el sistema,
    - requiere información perfecta previa.
- **Dinámica**:
    - decisiones tomadas en tiempo de ejecución,
    - más flexible, menos predecible.

2.5 Problemas clásicos de comunicación entre procesos (IPC)

Esta sección presenta **problemas modelo** usados para estudiar **sincronización y exclusión mutua**. No son ejercicios artificiales: representan situaciones reales de competencia por recursos y acceso a datos compartidos.

2.5.1 El problema de los filósofos comelones

Situación

- Cinco filósofos se sientan alrededor de una mesa.
- Cada uno alterna entre **pensar** y **comer**.
- Para comer necesita **dos tenedores**.
- Hay **un tenedor entre cada par de filósofos**.

Objetivo

Permitir que los filósofos coman y piensen:

- sin interbloqueo,
- sin inanición,
- permitiendo el mayor paralelismo posible.

Problemas de las soluciones simples

- Si todos toman un tenedor al mismo tiempo → **interbloqueo**.
- Si sueltan y reintentan todos juntos → **inanición**.

- Usar un mutex global evita errores, pero **sólo permite que coma uno a la vez**, desperdiciando recursos.

Idea de la solución correcta

- Cada filósofo tiene un **estado**: pensando, hambriento o comiendo.
- Un filósofo **sólo puede comer si sus dos vecinos no están comiendo**.
- Se usan:
    - un mutex para proteger el estado global,
    - un semáforo por filósofo para bloquearlo si no puede comer.

Resultado

- No hay interbloqueo.
- No hay inanición.
- Pueden comer varios filósofos al mismo tiempo si no son vecinos.

2.5.2 El problema de los lectores y escritores

Situación

Modela el acceso concurrente a una base de datos:

- **Lectores** sólo leen.
- **Escritores** modifican datos.

Reglas

- Varios lectores pueden leer al mismo tiempo.
- Si un escritor escribe, nadie más puede acceder.

Solución clásica

- Se permite que múltiples lectores entren juntos.
- El primer lector bloquea la base de datos.
- El último lector la libera.
- El escritor necesita acceso exclusivo.

Problema

- Si llegan lectores constantemente, un escritor puede quedar bloqueado indefinidamente → **inanición de escritores**.

Alternativa

- Dar prioridad a escritores.
- Evita inanición, pero reduce concurrencia y rendimiento.

UNIDAD 3 - ADMINISTRACIÓN DE MEMORIA

La **memoria principal (RAM)** es un recurso crítico que debe administrarse con cuidado. Aunque la memoria creció muchísimo con los años, **los programas crecen todavía más rápido**.
El sistema operativo crea **abstracciones de memoria** y las administra para:

- asignar memoria a los procesos,
- evitar interferencias,
- y usarla de forma eficiente.w

Las computadoras no tienen una sola memoria ideal, sino una **jerarquía de memoria**:

- caché: muy rápida, pequeña y volátil,
- memoria principal: más grande, velocidad media, volátil,
- disco: grande, lento y no volátil,
- Almacenamiento removible.

El **administrador de memoria** es la parte del sistema operativo que:

- lleva registro de qué memoria está en uso,
- asigna memoria a los procesos,
- la libera cuando ya no se necesita.

En este capítulo se estudia **la memoria principal**, desde esquemas simples hasta sofisticados.

3.1 Sin abstracción de memoria

Modelo

No existe ninguna abstracción: cada programa ve **directamente la memoria física**.

Una instrucción como:

```
MOV REGISTRO1, 1000
```

accede **a la dirección física 1000** de la RAM.

El programador ve la memoria como:

- direcciones desde 0 hasta un máximo,
- cada dirección corresponde a una celda de memoria.